

Estudo Dirigido · Guia 3

Movimentação e Input

Controle do Jogador pelo Teclado

Este guia continua a série e aborda a interatividade fundamental de qualquer jogo: como o jogador controla seu personagem. Vamos implementar detecção de teclado com movimento suave, contínuo e limitado pelas bordas da tela.

Objetivos de Aprendizagem

- Entender a diferença entre eventos pontuais e estado contínuo de teclas
- Usar `pygame.key.get_pressed()` para movimento fluido
- Implementar movimentação horizontal e vertical
- Aplicar limites de borda para impedir que o sprite saia da tela
- Montar um Game Loop completo com input e renderização

1. Os Três Pilares da Movimentação

Para que o jogador possa controlar seu personagem de forma responsiva, três elementos precisam trabalhar em conjunto:

1. Posição <code>player_x, player_y</code> Coordenadas atuais do sprite na tela	2. Velocidade <code>PLAYER_SPEED</code> Pixels deslocados por frame a cada input	3. Input <code>get_pressed()</code> Estado atual de todas as teclas a cada frame
--	---	---

1.1. Configuração das Variáveis

Antes do Game Loop, declare as variáveis de posição e velocidade. A velocidade é definida em pixels por frame — quanto maior o valor, mais rápido o personagem se move:

```
# Posição inicial: centro horizontal, perto do fundo
player_x = (SCREEN_WIDTH // 2) - (PLAYER_IMAGE.get_width() // 2)
player_y = SCREEN_HEIGHT - PLAYER_IMAGE.get_height() - 10

# Velocidade em pixels por frame (ajuste conforme necessário)
PLAYER_SPEED = 5
```

💡 Velocidade e FPS

Com `PLAYER_SPEED = 5` e `clock.tick(60)`, o jogador percorre até 300 pixels por segundo (5×60). Para velocidade independente do FPS, use delta time: $\text{distancia} = \text{velocidade} * \text{dt}$, onde $\text{dt} = \text{clock.tick}(60) / 1000$.

2. Detecção de Teclado

2.1. Dois Métodos: Eventos vs. Estado Contínuo

O Pygame oferece duas formas de capturar o teclado. Entender a diferença é essencial para escolher a abordagem correta:

❌ Evento (KEYDOWN)

```
for event in pygame.event.get():
    if event.type == pygame.KEYDOWN:
        if event.key == pygame.K_LEFT:
            player_x -= SPEED
```

Disparado apenas UMA VEZ ao pressionar. O movimento trava até soltar e pressionar de novo.

✅ Estado contínuo (get_pressed)

```
keys = pygame.key.get_pressed()
if keys[pygame.K_LEFT]:
    player_x -= SPEED
```

Verificado a CADA frame. Enquanto a tecla estiver pressionada, o sprite se move continuamente.

⚠ Quando usar cada um?

Use o loop de eventos (KEYDOWN) para ações pontuais: disparar, pausar, confirmar. Use `get_pressed()` para movimentação contínua. Ambos podem coexistir no mesmo Game Loop.

2.2. Movimento nos Quatro Eixos

Com `pygame.key.get_pressed()`, verificamos o estado de cada tecla de direção e ajustamos as coordenadas do jogador. Este trecho fica dentro da seção "2. Atualizar o Estado" do Game Loop:

```
# Obtém o estado de TODAS as teclas neste frame
keys = pygame.key.get_pressed()

# Movimento Horizontal
if keys[pygame.K_LEFT]:
    player_x -= PLAYER_SPEED # move para a esquerda
if keys[pygame.K_RIGHT]:
    player_x += PLAYER_SPEED # move para a direita

# Movimento Vertical
if keys[pygame.K_UP]:
    player_y -= PLAYER_SPEED # move para cima (Y decresce!)
if keys[pygame.K_DOWN]:
```

```
player_y += PLAYER_SPEED    # move para baixo
```

💡 Eixo Y invertido

No Pygame, Y = 0 é o topo da tela e aumenta para baixo. Por isso, para mover o sprite para cima, subtraímos da posição Y (player_y -= speed).

3. Limitação de Bordas

Sem restrições, o sprite pode sair completamente da janela. Após atualizar a posição, aplique verificações de limite para que o jogador nunca ultrapasse as quatro bordas da tela:

← **Esquerda**
`player_x < 0`
→ `player_x = 0`

↑ **Superior**
`player_y < 0`
→ `player_y = 0`

→ **Direita**
`player_x > W - img.w`
→ `player_x = W - img.w`

3.1. Código de Limitação

Adicione este bloco logo após a atualização de posição pelas teclas, ainda dentro da seção de atualização do Game Loop:

```
# — Limites das bordas —————

# Borda esquerda
if player_x < 0:
    player_x = 0

# Borda direita (desconta a largura do sprite)
if player_x > SCREEN_WIDTH - PLAYER_IMAGE.get_width():
    player_x = SCREEN_WIDTH - PLAYER_IMAGE.get_width()

# Borda superior
if player_y < 0:
    player_y = 0

# Borda inferior (desconta a altura do sprite)
if player_y > SCREEN_HEIGHT - PLAYER_IMAGE.get_height():
    player_y = SCREEN_HEIGHT - PLAYER_IMAGE.get_height()
```

💡 Por que descontar o tamanho do sprite?

A posição (player_x, player_y) representa o canto superior ESQUERDO do sprite, não seu centro. Para que a borda DIREITA do sprite não ultrapasse a tela, o limite máximo é SCREEN_WIDTH - largura_do_sprite (e analogamente para a borda inferior).

3.2. Versão Compacta com max() e min()

Uma alternativa mais concisa usa as funções embutidas do Python para restringir o valor em uma única linha por eixo:

```
# Versão compacta equivalente ao bloco acima
player_x = max(0, min(player_x, SCREEN_WIDTH - PLAYER_IMAGE.get_width()))
player_y = max(0, min(player_y, SCREEN_HEIGHT - PLAYER_IMAGE.get_height()))
```

4. Game Loop Completo

O código abaixo integra todos os conceitos dos Guias 1, 2 e 3 em um programa funcional: inicialização, carregamento de sprite, movimentação contínua com teclado e limitação de bordas:

```
import pygame
import sys

# — Inicialização —
pygame.init()
SCREEN_WIDTH, SCREEN_HEIGHT = 800, 600
SCREEN = pygame.display.set_mode((SCREEN_WIDTH, SCREEN_HEIGHT))
pygame.display.set_caption("Guardião da Força")
clock = pygame.time.Clock()

# — Sprite —
try:
    PLAYER_IMAGE = pygame.image.load("nave.png").convert_alpha()
except pygame.error:
    PLAYER_IMAGE = pygame.Surface((50, 50))
    PLAYER_IMAGE.fill((0, 180, 255))

# — Estado inicial —
player_x = (SCREEN_WIDTH // 2) - (PLAYER_IMAGE.get_width() // 2)
player_y = SCREEN_HEIGHT - PLAYER_IMAGE.get_height() - 10
PLAYER_SPEED = 5
BACKGROUND_COLOR = (0, 0, 20) # azul-noturno

# — Game Loop —
running = True
while running:

    # 1. Eventos
```

```
for event in pygame.event.get():
    if event.type == pygame.QUIT:
        running = False
    if event.type == pygame.KEYDOWN:
        if event.key == pygame.K_ESCAPE:
            running = False

# 2. Atualização - Input contínuo
keys = pygame.key.get_pressed()
if keys[pygame.K_LEFT]: player_x -= PLAYER_SPEED
if keys[pygame.K_RIGHT]: player_x += PLAYER_SPEED
if keys[pygame.K_UP]: player_y -= PLAYER_SPEED
if keys[pygame.K_DOWN]: player_y += PLAYER_SPEED

# 2b. Limites das bordas
player_x = max(0, min(player_x, SCREEN_WIDTH - PLAYER_IMAGE.get_width()))
player_y = max(0, min(player_y, SCREEN_HEIGHT - PLAYER_IMAGE.get_height()))

# 3. Renderização
SCREEN.fill(BACKGROUND_COLOR)
SCREEN.blit(PLAYER_IMAGE, (player_x, player_y))
pygame.display.flip()
clock.tick(60)

pygame.quit()
sys.exit()
```

5. Referência de Teclas e Métodos

Constantes de Teclado Comuns

Constante Pygame	Tecla
<code>pygame.K_LEFT / K_RIGHT</code>	Teclas de seta esquerda / direita
<code>pygame.K_UP / K_DOWN</code>	Teclas de seta para cima / baixo
<code>pygame.K_a / K_d</code>	Teclas A e D (alternativa WASD)
<code>pygame.K_w / K_s</code>	Teclas W e S (alternativa WASD)
<code>pygame.K_SPACE</code>	Barra de espaço (ex: disparar)
<code>pygame.K_ESCAPE</code>	Tecla ESC (ex: pausar / sair)
<code>pygame.K_RETURN</code>	Tecla Enter / Return

6. Exercícios Práticos

Reforce os conceitos com os desafios abaixo, em ordem crescente de dificuldade:

Exercício 1 — Ajuste de velocidade

Mude `PLAYER_SPEED` para 3 e depois para 10. Compare a sensação de controle em cada valor.

Exercício 2 — Controles WASD

Adicione suporte às teclas W, A, S, D como alternativa às setas. O jogador deve poder usar qualquer um dos dois esquemas.

Exercício 3 — Movimento diagonal

Observe que pressionar ← e ↑ ao mesmo tempo move o sprite na diagonal. Qual é a velocidade resultante? Como normalizá-la para manter a mesma velocidade de 5 px/frame em qualquer direção?

Exercício 4 — Limite apenas horizontal

Remova os limites verticais e deixe apenas os horizontais. Use isso para criar um jogo onde o jogador pode sair pelo topo e reaparecer pela base.

Exercício 5 — Velocidade progressiva

Implemente aceleração: o sprite começa devagar e aumenta a velocidade enquanto a tecla fica pressionada, até um máximo de 10 px/frame.

7. Próximos Passos

Com movimentação e input dominados, você está pronto para os próximos desafios da série:

Guia 4 — Colisões e Grupos

- `pygame.Rect` para detecção de colisão
- `colliderect()` entre dois sprites
- `pygame.sprite.Group` para múltiplos objetos

Guia 5 — Sons e Pontuação

- `pygame.mixer` para efeitos sonoros
- `pygame.font` para texto na tela
- Placar, vidas e estados de jogo